

```
In [1]: import pandas as pd
import sympy as sp
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
from numpy import linalg as LA
```

**NOTE TO GRADER:** I built this notebook using [Marimo](#) as I manage a Git repository of all Python notebooks for personal and academic purposes. Jupyter and Git mix like oil and water. This notebook is near identical to the Jupyter notebook with a few minor changes (that should not be of consequence but I have noted them in case they are).

- `from sympy import *` has been transformed into `import sympy as sp` due to Marimo not supporting wildcard imports.
- All imports are done in a singular setup cell instead of throughout the code. (this also pleases the CS major side of me where all imports should always be done at the top of your file)
- Some formatting regarding printing is slightly different than what would be expected from Jupyter Lab.
- In Marimo, global variables cannot be defined more than once. So any place that tries to redefine variables with a name defined in another cell is wrapped in a `def _(): ... _()`

## 1.1 Motivating Example: Penguin Data

To begin our lab section, we will discuss a specific task that you may encounter as a data scientist: classifying penguin species from different features. Without knowing what species each penguin is in a given row, we can identify different clusters in the data.

```
In [2]: # Download Data
# Github: https://github.com/allisonhorst/palmerpenguins/blob/main/README.md

# raw link
data = pd.read_csv('https://raw.githubusercontent.com/MMiDS-textbook/MMiDS-textbook')
```

```
In [3]: # Show first 5 rows of data
data.head() # note NaNs
```

```
Out[3]:
```

|          | bill_length_mm | bill_depth_mm | flipper_length_mm | body_mass_g |
|----------|----------------|---------------|-------------------|-------------|
| <b>0</b> | 39.1           | 18.7          | 181.0             | 3750.0      |
| <b>1</b> | 39.5           | 17.4          | 186.0             | 3800.0      |
| <b>2</b> | 40.3           | 18.0          | 195.0             | 3250.0      |
| <b>3</b> | NaN            | NaN           | NaN               | NaN         |
| <b>4</b> | 36.7           | 19.3          | 193.0             | 3450.0      |

```
In [4]: # TODO: count missing values per column
print(data.isnull().sum())
```

```
bill_length_mm      2
bill_depth_mm       2
flipper_length_mm   2
body_mass_g         2
dtype: int64
```

```
In [5]: # TODO: print shape and interpret (n rows, d columns)
print("Shape:", data.shape)
print(f"There are {data.shape[0]} rows (samples) and {data.shape[1]} columns (features)")
```

```
Shape: (344, 4)
There are 344 rows (samples) and 4 columns (features).
```

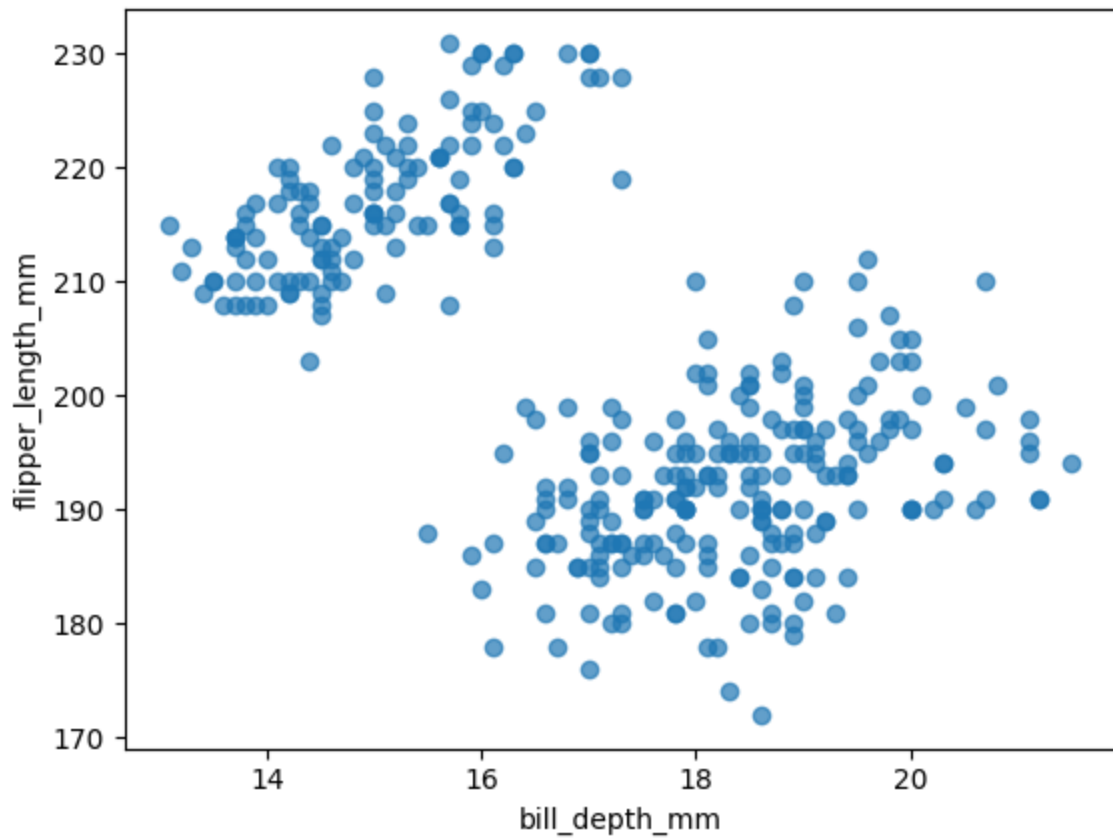
```
In [6]: # TODO: convert DataFrame to numpy array X
X = data.dropna().to_numpy()

print("X shape:", X.shape)
print(X)
```

```
X shape: (342, 4)
[[ 39.1  18.7 181. 3750.]
 [ 39.5  17.4 186. 3800.]
 [ 40.3  18.  195. 3250.]
 ...
 [ 50.4  15.7 222. 5750.]
 [ 45.2  14.8 212. 5200.]
 [ 49.9  16.1 213. 5400.]]
```

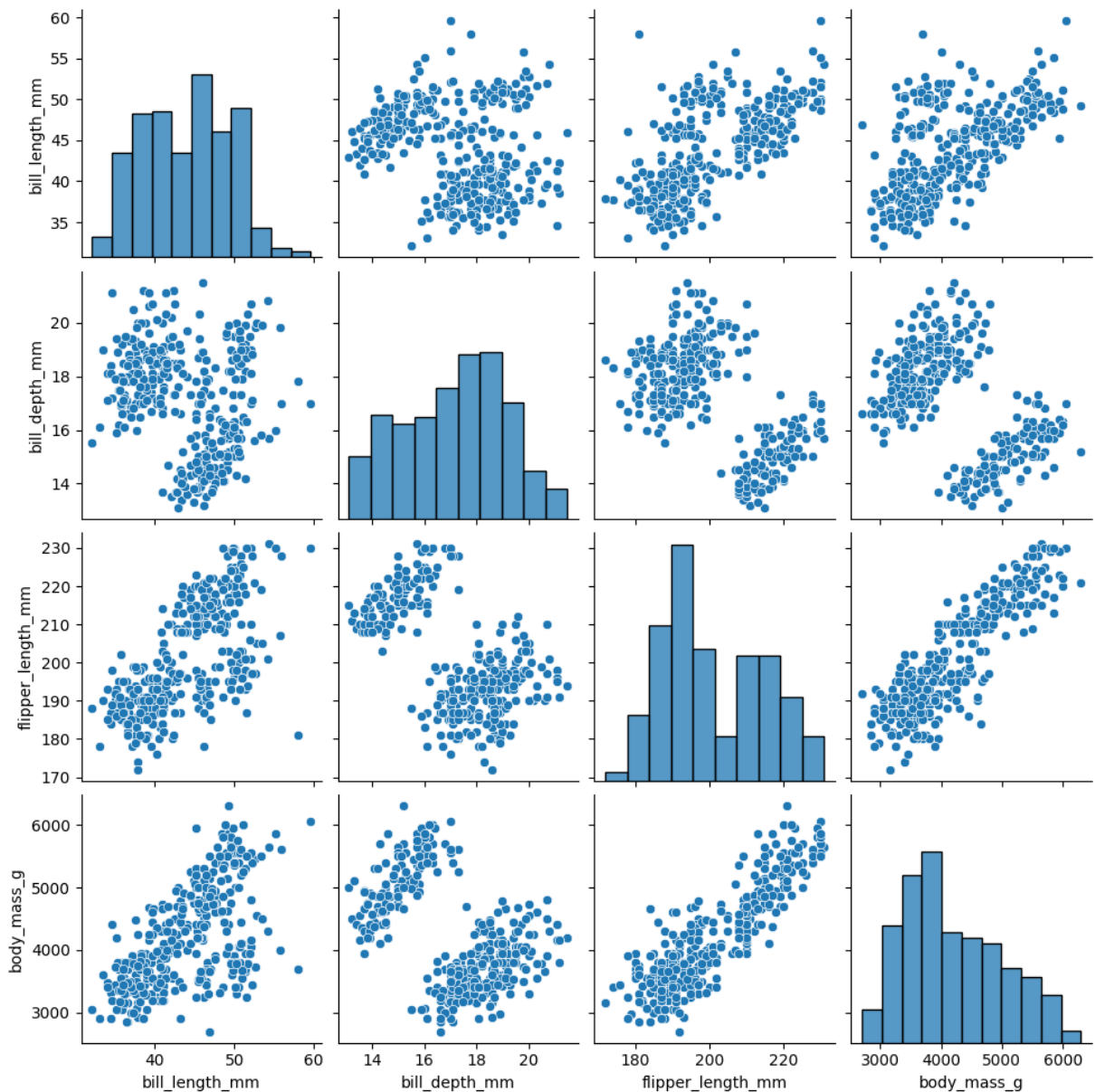
```
In [7]: # Plot 2 columns
# TODO: make a scatter plot of column 1 vs column 2 (use X slicing)
plt.scatter(X[:, 1], X[:, 2], alpha=0.7)

plt.xlabel('bill_depth_mm')
plt.ylabel('flipper_length_mm')
plt.show()
```



```
In [8]: # Plot all column pairs
# TODO: pairplot the dataframe (numeric columns only if needed)
# We include 'species' for the hue to see separation, even though the math part foc
sns.pairplot(data)

plt.show()
# which feature pairs look usefull for separating the species?
```



## 1.2.1 Vectors and Matrices (Linear Algebra)

```
In [9]: # Numpy is the popular math module in Python
# A common alias is np
# we can create an array using the "array" function
u = np.array([1., 3., 5., 7.])

# we can view the array by printing it to the screen
# note: although vectors are typically notated as columns, python prints them horiz
print(u)
```

```
[1.  3.  5.  7.]
```

```
In [10]: # TODO: print the first and second elements of u (remember Python is 0-indexed)

# arrays can be indexed using brackets, Python indexing starts at 0
print("First element:", u[0])
print("Second element:", u[1])
```

First element: 1.0  
Second element: 3.0

```
In [11]: # numpy has a linear algebra sub module that we can use for linear algebra computat
# the documentation can be found at: https://numpy.org/doc/stable/reference/routine

# calculate the norm of our vector u
# TODO: compute ||u||_2 using numpy.linalg
norm_u = LA.norm(u)
print("L2 Norm of u:", norm_u)
```

L2 Norm of u: 9.16515138991168

Sanity Check: Does this answer match with what you get using the formula for the Norm?

Hint:  $\|u\|_2 = \sqrt{\sum_{i=1}^d u_i^2}$

```
In [12]: # here's a way to check using python
check = np.sqrt(
    np.sum(u ** 2)
)
print("Manual calculation:", check)

# the vectorization of numpy allows us to perform operations on arrays without need
```

Manual calculation: 9.16515138991168

## $\ell_1$ vs $\ell_2$ Norm

```
In [13]: # Vectors v and w
v = np.array([1, 2, 3, 4])
w = np.array([0, 1, 0, 1])
```

```
In [14]: # Let's calculate the L1 norm of v and w
print('V L1 Norm:', LA.norm(v, ord = 1))
print('W L1 Norm:', LA.norm(w, ord = 1), '\n')

# now the L2 norm of v and w
# TODO: print L1 and L2 norms of v and w using LA.norm(..., ord=?)
print('V L2 Norm:', LA.norm(v, ord = 2))
print('W L2 Norm:', LA.norm(w, ord = 2))
```

V L1 Norm: 10.0

W L1 Norm: 2.0

V L2 Norm: 5.477225575051661

W L2 Norm: 1.4142135623730951

## P Norm Investigation

How does the P norm value change as P changes?

## The $p$ -norm of a vector

For a vector  $u = (u_1, u_2, \dots, u_d)$  and  $p \geq 1$ , the  $p$ -norm is defined as

$$\|u\|_p = \left( \sum_{i=1}^d |u_i|^p \right)^{1/p}.$$

As  $p \rightarrow \infty$ , the  $p$ -norm converges to the infinity norm:

$$\|u\|_\infty = \max_{1 \leq i \leq d} |u_i|.$$

## Visualizing the $p$ -norm as “circles”

A **norm**  $\|\cdot\|$  defines a notion of length. Once we have a norm, we can define a distance between two points  $x, c \in \mathbb{R}^d$  by:

$$d_p(x, c) = \|x - c\|_p.$$

In ordinary Euclidean geometry, the set of points at distance  $r$  from the origin is a **circle**:

$$\{x \in \mathbb{R}^2 : \|x\|_2 = r\}.$$

But if we change the norm, the “circle” changes shape. For  $p \geq 1$ :

$$\|x\|_p = (|x_1|^p + |x_2|^p)^{1/p}.$$

We will plot **iso-distance contours** (level sets) for several  $p$  values:

$$\{(x_1, x_2) : \|(x_1, x_2)\|_p = r\}.$$

**Key idea:** changing  $p$  changes what “equally far” means.

- $p = 2$  (**L2 / Euclidean**) → round circles
- $p = 1$  (**L1**) → diamond-shaped contours
- $p \rightarrow \infty$  ( $L_\infty$ ) → square-shaped contours (distance is dominated by the largest coordinate)

This matters for the rest of the notebook because “closeness” between data points (and later, model penalties) depends on the choice of norm.

```
In [15]: def lp_norm_2d(X, Y, p):  
    """Compute ||(X,Y)||_p on a grid."""  
    if np.isinf(p):  
        return np.maximum(np.abs(X), np.abs(Y))  
    return (np.abs(X)**p + np.abs(Y)**p)**(1/p)  
  
# Grid over R^2
```

```

xs = np.linspace(-2, 2, 401)
ys = np.linspace(-2, 2, 401)
X_grid, Y_grid = np.meshgrid(xs, ys)

# Norm choices to compare
ps = [1, 2, 4, np.inf]
levels = [0.5, 1.0, 1.5] # radii r

fig, axes = plt.subplots(2, 2, figsize=(9, 9))
axes = axes.ravel()

for ax, p in zip(axes, ps):
    Z = lp_norm_2d(X_grid, Y_grid, p)

    cs = ax.contour(X_grid, Y_grid, Z, levels=levels, linewidths=2)
    ax.clabel(cs, inline=True, fontsize=9)

    # Axes lines for reference
    ax.axhline(0, linewidth=1)
    ax.axvline(0, linewidth=1)

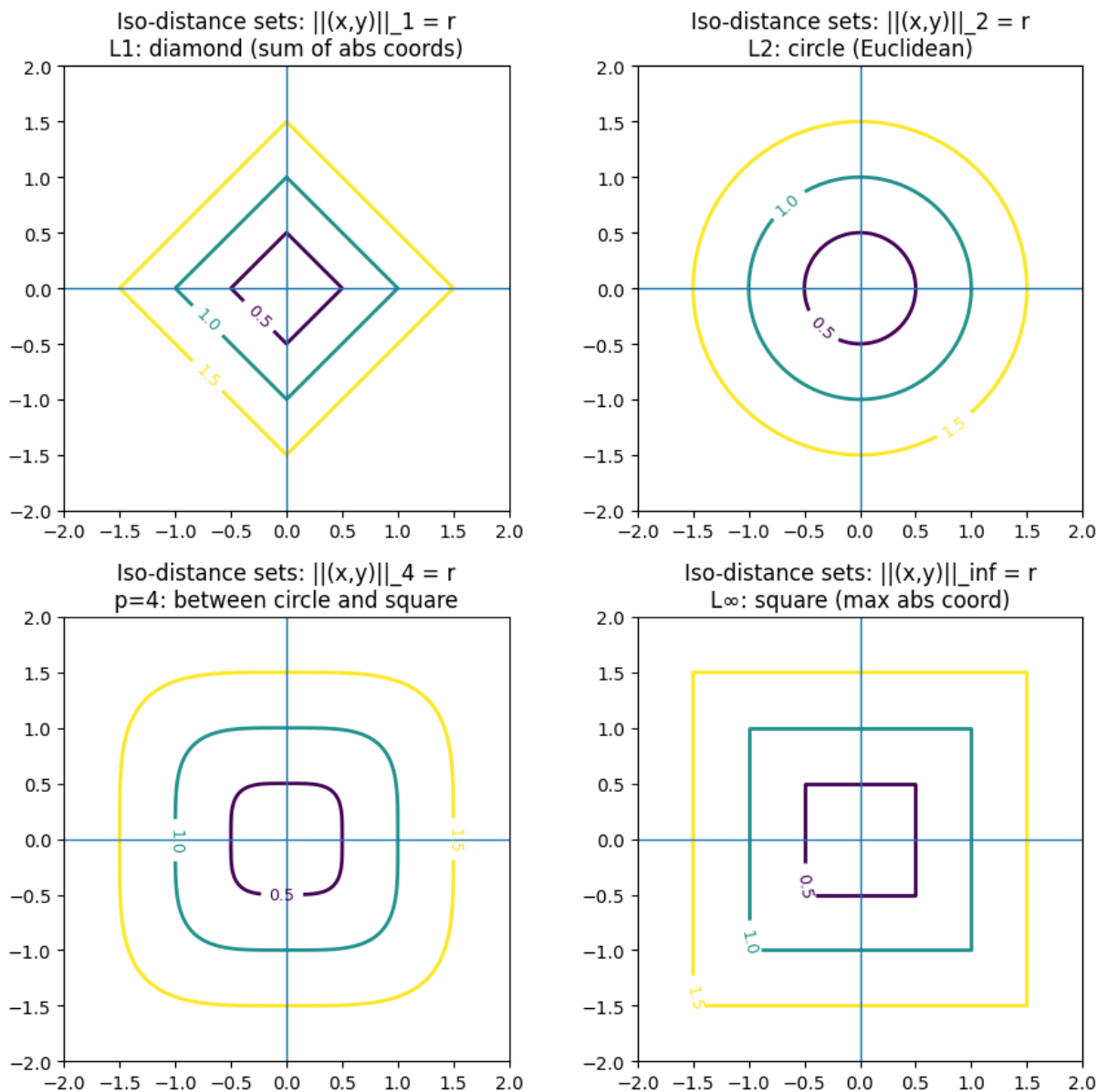
    # Equal scaling so shapes aren't distorted
    ax.set_aspect('equal', 'box')
    ax.set_xlim(-2, 2)
    ax.set_ylim(-2, 2)

    # Title + quick interpretation
    if p == 1:
        subtitle = "L1: diamond (sum of abs coords)"
    elif p == 2:
        subtitle = "L2: circle (Euclidean)"
    elif p == 4:
        subtitle = "p=4: between circle and square"
    else:
        subtitle = "L∞: square (max abs coord)"

    ax.set_title(f"Iso-distance sets:  $\|(x,y)\|_{\{p\}} = r \backslash n \{ \text{subtitle} \}")

fig.suptitle("Changing p changes the geometry of distance", fontsize=14)
plt.tight_layout()
plt.show()$ 
```

## Changing p changes the geometry of distance



Exercise prompt:

"Which norm would treat outliers along one feature axis as most influential? (hint:  $L_\infty$ )"

"Which norm would make axis-aligned differences count more than diagonal ones? (hint:  $L_1$ )"

**Key takeaway:** The value of a norm depends on how it weights large vs small components. As  $p$  increases, the largest component dominates the norm. This is why different norms lead to different notions of distance and different regularization behavior in machine learning.

A norm measures size, a  $p$ -norm is a specific way to measure size, and regularization works by penalizing the norm of the model parameters—so choosing  $p$  directly determines what kinds of solutions the model prefers.



## Vector addition, scalar multiplication, euclidean distance, and transpose and indexing of matrices

```
In [16]: # Vector Addition
v_1 = np.array([1, 2, 3, 4])
w_1 = np.array([0, 1, 0, 1])

# we can add two vectors
# TODO: compute v + w and print it
print("v + w:", v_1 + w_1)
```

v + w: [1 3 3 5]

```
In [17]: # Scalar Multiplication
print(2 * w_1)
```

[0 2 0 2]

```
In [18]: # Find Euclidean distance

# initializing vectors as numpy arrays
point1 = np.array((1, 2, 3))
point2 = np.array((1, 1, 1))

# calculating Euclidean distance using linalg.norm()
dist = np.linalg.norm(point1 - point2)

# printing Euclidean distance
print("Numpy Distance:", dist)
```

Numpy Distance: 2.23606797749979

```
In [19]: # Sanity check
our_dist = np.sqrt((1-1)**2 + (2-1)**2 + (3-1)**2)

# TODO: confirm it matches the norm-based computation
print("Manual Distance:", our_dist)
print("Match:", np.isclose(dist, our_dist))
```

Manual Distance: 2.23606797749979

Match: True

## Matrices

```
In [20]: # can create 2 different vectors
u_1 = np.array([1.0, 3.0, 5.0, 7.0])
v_2 = np.array([2.0, 4.0, 6.0, 8.0])
X_1 = np.stack((u_1, v_2), axis=0)
# using np.stack, we can put the two vectors together
# the axis parameter changes what direction they are stacked together
print(X_1, '\n')

# we can also use np.vstack and np.hstack
# TODO: use np.vstack and np.hstack
v_stack = np.vstack((u_1, v_2))
```

```

h_stack = np.hstack((u_1, v_2))
print("Vertical Stack:\n", v_stack)
print("Horizontal Stack:\n", h_stack)

print("Shape of X_1:", X_1.shape)

```

```

[[1. 3. 5. 7.]
 [2. 4. 6. 8.]]

```

Vertical Stack:

```

[[1. 3. 5. 7.]
 [2. 4. 6. 8.]]

```

Horizontal Stack:

```

[1. 3. 5. 7. 2. 4. 6. 8.]

```

Shape of X\_1: (2, 4)

```

In [21]: # combining the two vectors in one to another vector
Y_1 = np.array([u_1, v_2])
print(Y_1)

```

```

[[1. 3. 5. 7.]
 [2. 4. 6. 8.]]

```

```

In [22]: # we can access using bracket indexing: [row, column]
print(Y_1[0, 0])
print(Y_1[0, 1])

```

```

1.0
3.0

```

```

In [23]: # Transpose of Matrix
V = np.array(
    [
        [1, 2, 3],
        [4, 5, 6]
    ]
)
# all numpy arrays have a shape attribute that we can access
print('Shape of V:', V.shape, '\n')

# we can use the transpose function
VT = np.transpose(V)
print(r'V^{T}:', '\n', VT, '\n')

# or we can use .T
print(r'V^{T}:', '\n', V.T, '\n')

# note: although vectors are typically notated as columns, python prints them horiz
print(VT.shape)

```

Shape of V: (2, 3)

```
V{T}:  
[[1 4]  
 [2 5]  
 [3 6]]
```

```
V{T}:  
[[1 4]  
 [2 5]  
 [3 6]]
```

(3, 2)

```
In [24]: # Transpose of a Matrix without Numpy  
  
def transpose(matrix):  
    """Transposes a given matrix."""  
  
    # Create a new matrix with swapped dimensions  
    transposed_matrix = [[0 for _ in range(len(matrix))] for _ in range(len(matrix[0]))]  
  
    # Fill the transposed matrix  
    for i in range(len(matrix)):  
        for j in range(len(matrix[0])):  
  
            # swapping the j,i entry of the transpose to the i,j entry of the original matrix  
            transposed_matrix[j][i] = matrix[i][j]  
  
    return transposed_matrix  
  
matrix = V  
print(transpose(matrix))
```

```
[[np.int64(1), np.int64(4)], [np.int64(2), np.int64(5)], [np.int64(3), np.int64(6)]]
```

## Matrix Multiplication

Note: Order matters! The inside dimensions of the multiplication must match.

i.e. We can multiply a 2x3 matrix with a 3x4, but we cannot multiply a 3x4 matrix with a 2x3 matrix.

A valid matrix multiplication:

$$A_{2 \times 3} B_{3 \times 4}$$

A not valid matrix multiplication:

$$B_{3 \times 4} A_{2 \times 3}$$

```
In [25]: ### we can multiply a vector and a matrix together  
  
# Let's make an array to multiply with our V matrix  
# it must be 1x2 or 3x1 to be multiplied with a 2x3 matrix
```

```

v1 = np.array([1, 2])

# the transpose makes it a column vector
v2 = np.array([1, 2, 3]).T

# use the matmul function to do matrix multiplication, order matters!
print('v1 dot V:', np.matmul(v1, V))
print('V dot v2:', np.matmul(V, v2))

```

```

v1 dot V: [ 9 12 15]
V dot v2: [14 32]

```

```

In [26]: # how about matrix multiplication?
# define a valid matrix
U = np.array(
    [
        [5, 6],
        [2, 7],
        [10, 3]
    ]
)

print('U Matrix', '\n', U, '\n')

# use matmul again

# TODO: compute U @ V and V @ U
UV = np.matmul(U, V)
VU = np.matmul(V, U)

print('U dot V', '\n', UV, '\n')
print('V dot U', '\n', VU)

```

```

U Matrix
[[ 5  6]
 [ 2  7]
 [10  3]]

```

```

U dot V
[[29 40 51]
 [30 39 48]
 [22 35 48]]

```

```

V dot U
[[39 29]
 [90 77]]

```

## Frobenious

This norm is applied to an  $m \times n$  matrix. The Frobenious norm is defined as:

$$\|A\|_F = \left( \sum_{i=1}^n \sum_{j=1}^m A_{i,j}^2 \right)^{\frac{1}{2}}$$

```
In [27]: A = np.array(  
    [  
        [1., 0.],  
        [0., 1.],  
        [0., 0.]  
    ]  
    )  
    print(A)
```

```
[[1. 0.]  
 [0. 1.]  
 [0. 0.]
```

```
In [28]: print("Frobenius Norm:", LA.norm(A))
```

Frobenius Norm: 1.4142135623730951

We expect this value to be  $(1^2 + 0^2 + 0^2 + 1^2 + 0^2 + 0^2)^{\frac{1}{2}} = (2)^{\frac{1}{2}} = \sqrt{2}$ . Does our answer match?

```
In [29]: # TODO: verify it matches sqrt(2)  
    calculated_norm = LA.norm(A)  
    expected_norm = np.sqrt(2)  
    print(f"Calculated: {calculated_norm}")  
    print(f"Expected: {expected_norm}")  
    print(f"Match: {np.isclose(calculated_norm, expected_norm)}")
```

Calculated: 1.4142135623730951

Expected: 1.4142135623730951

Match: True

## 1.2.2 Plotting Functions

For now, we will just look at how to plot different functions using the matplotlib package.

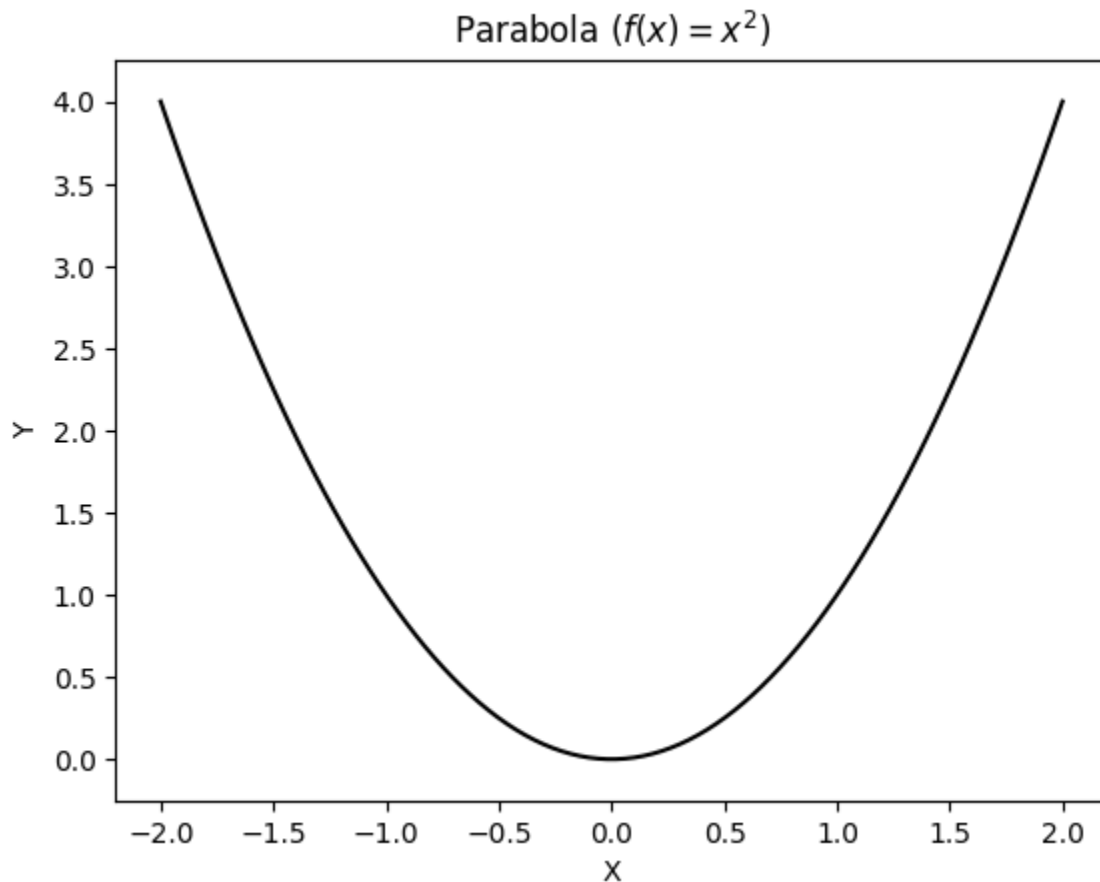
```
In [30]: def _():
# matplotlib is the main plotting library in Python
x = np.linspace(-2, 2, 100)

# TODO: compute y for f(x)=x^2
y = x**2

plt.plot(x, y, c='k')
# linspace creates an evenly spaced array
plt.ylim(-0.25, 4.25)
plt.xlabel('X')

plt.ylabel('Y')
# we can plot using plt
plt.title('Parabola ($f(x) = x^2$)')
# optional: Titles
return plt.show()

_()
```



```
In [31]: def _():
# create the values for x and calculate the y values using the exponential func
x_1 = np.linspace(-2, 2, 100)

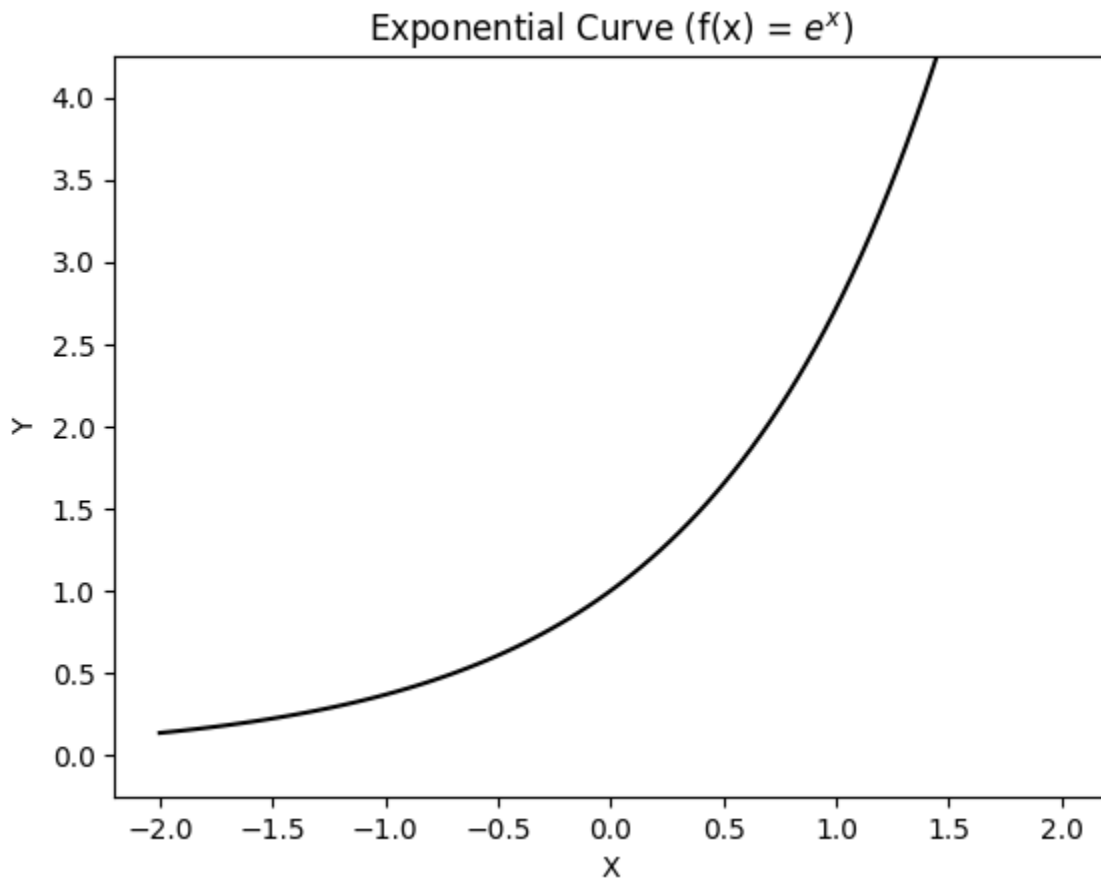
# TODO: compute y = exp(x)
y = np.exp(x_1)
```

```

plt.plot(x_1, y, c='k')
# plot them, c = 'k' means color of the line is black
plt.ylim(-0.25, 4.25)
plt.xlabel('X')
plt.ylabel('Y')
# optional: Titles
plt.title('Exponential Curve (f(x) =  $e^x$ )')
return plt.show()

```

\_( )



```

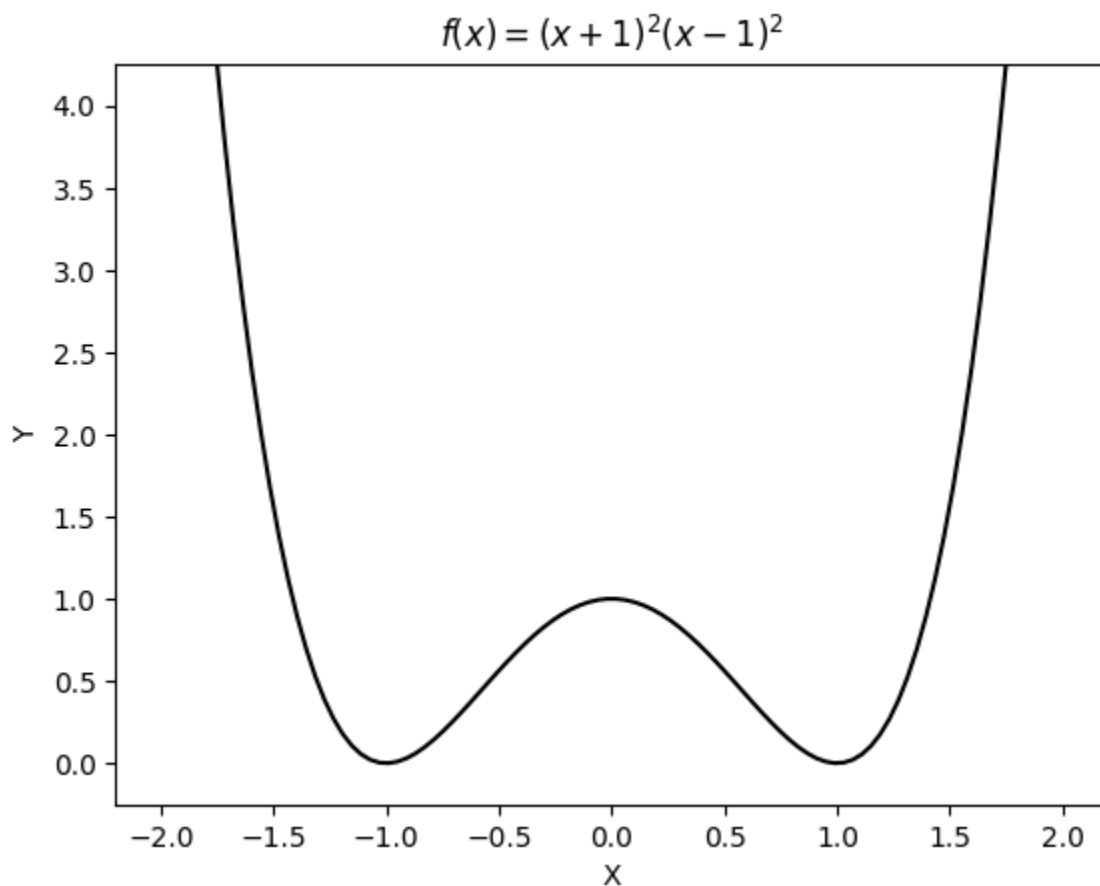
In [32]: def _( ):
x_2 = np.linspace(-2, 2, 100)

# create a new function
# TODO: compute y = (x+1)^2 (x-1)^2
y = ((x_2 + 1)**2) * ((x_2 - 1)**2)

plt.plot(x_2, y, c='k')
plt.ylim(-0.25, 4.25)
plt.xlabel('X')
plt.ylabel('Y')
plt.title('$f(x) = (x + 1)^2 (x-1)^2$')
# optional: Titles
# Tip: You can use latex in titles if you
# place it in string formatted as r''
return plt.show()

```

\_()



## Sympy

Another package we can use is Sympy for symbolic python. The documentation is found here: <https://docs.sympy.org/latest/index.html>. The documentatation on using matrices with sympy can be found here: <https://docs.sympy.org/latest/tutorials/intro-tutorial/matrices.html>.

```
In [33]: # Import sympy package
```

```
# Makes sympy formatted nicely  
sp.init_printing(use_unicode=True)
```

```
In [34]: # Matrix in nice display
```

```
A_1 = sp.Matrix([[1, 0, -2, -1], [2, 1, 0, -1], [-1, 3, -1, -2]])  
A_1
```

```
Out[34]: 
$$\begin{bmatrix} 1 & 0 & -2 & -1 \\ 2 & 1 & 0 & -1 \\ -1 & 3 & -1 & -2 \end{bmatrix}$$

```

```
In [35]: # Reduced Row Echelon Form
```



```
A_1.rref()
```

```
Out[35]:
```

$$\left( \begin{bmatrix} 1 & 0 & 0 & -\frac{1}{5} \\ 0 & 1 & 0 & -\frac{3}{5} \\ 0 & 0 & 1 & \frac{2}{5} \end{bmatrix}, (0, 1, 2) \right)$$